CS300

Elements of Software Engineering

Week 4: Requirements Engineering

Nguyễn Thị Minh Tuyền · Faculty of Information Technology, HCMUS

Topics Covered

1. Functional and non-functional requirements
2. Requirements engineering processes
3. Requirements elicitation and analysis
4. Requirements specification
5. Requirements validation
6. Requirements management

Definition

- **What is a Requirement?**
  - **It may range from:**
    - a high-level abstract statement of a service or of a system constraint, to
    - a detailed mathematical functional specification.
  - **Requirements may serve a dual function:**
    - May be the basis for a bid for a contract — therefore must be open to interpretation;
    - May be the basis for the contract itself — therefore must be defined in detail;
    - Both these statements may be called requirements.
- **Requirements Engineering**
  - The process of establishing the services that a customer requires from a system and the constraints under which it operates and is developed.

Types of Requirement

- **User requirements**
  - Statements in natural language plus diagrams of the services the system provides and its operational constraints.
  - Written for customers.
- **System requirements**
  - A structured document setting out detailed descriptions of the system's functions, services and operational constraints.
  - Defines what should be implemented so may be part of a contract between client and contractor.

Examples of User and System Requirements

- **User requirement definition**
  - 1. The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.
- **System requirements specification**
  - 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
  - 1.2 The system shall automatically generate the report for printing after 17:30 on the last working day of the month.
  - 1.3 A report shall be created for each clinic listing drug names, prescriptions, doses and total cost.
  - 1.4 Separate reports shall be created for each dose unit if drugs are available in different doses.
  - 1.5 Access to cost reports shall be restricted to authorized users on a management access control list.

System Stakeholders

- Any person or organization who is affected by the system in some way and so who has a legitimate interest.
- **Stakeholder types**
  - End users
  - System managers
  - System owners
  - External stakeholders

Example: Stakeholders in the Mentcare System

- **Patients** whose information is recorded in the system.
- **Doctors** who are responsible for assessing and treating patients.
- **Nurses** who coordinate consultations with doctors and administer treatments.
- **Medical receptionists** who manage patients' appointments.
- **IT staff** who are responsible for installing and maintaining the system.
- **A medical ethics manager** who must ensure the system meets ethical guidelines for patient care.
- **Health care managers** who obtain management information from the system.
- **Medical records staff** who ensure system information is maintained, preserved, and properly recorded.

Agile Methods and Requirements

- Many agile methods argue that producing detailed system requirements is a waste of time as requirements change so quickly.
- The requirements document is therefore always out of date.
- Agile methods usually use incremental requirements engineering and may express requirements as ‘user stories’.
- This is practical for business systems but problematic for systems that require pre-delivery analysis (e.g. critical systems) or systems developed by several teams.

Topics Covered

1. **Functional and non-functional requirements**
2. Requirements engineering processes
3. Requirements elicitation and analysis
4. Requirements specification
5. Requirements validation
6. Requirements management

Functional and Non-Functional Requirements

- **Functional requirements**
  - Statements of services the system should provide, how it should react to particular inputs, and how it should behave in particular situations.
  - May state what the system should not do.
- **Non-functional requirements**
  - Constraints on the services or functions offered by the system, such as timing constraints, constraints on the development process, standards, etc.
  - Often apply to the system as a whole rather than individual features or services.
- **Domain requirements**
  - Constraints on the system from the domain of operation.

Functional Requirements

- Describe functionality or system services.
- **Depend on:**
  - the type of software,
  - expected users, and
  - general approach taken by the organization.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.

Example: Functional Requirements for the Mentcare System

- A user shall be able to search the appointments lists for all clinics.
- The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
- Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.

Requirements Imprecision

- Problems arise when requirements are not precisely stated.
- Ambiguous requirements may be interpreted in different ways by developers and users.
- **Example: Consider the term 'search' in the requirement: "**A user shall be able to search the appointments lists for all clinics.**"**
  - **User intention:** search for a patient name across all appointments in all clinics.
  - **Developer interpretation:** search for a patient name in an individual clinic; user chooses clinic, then searches.

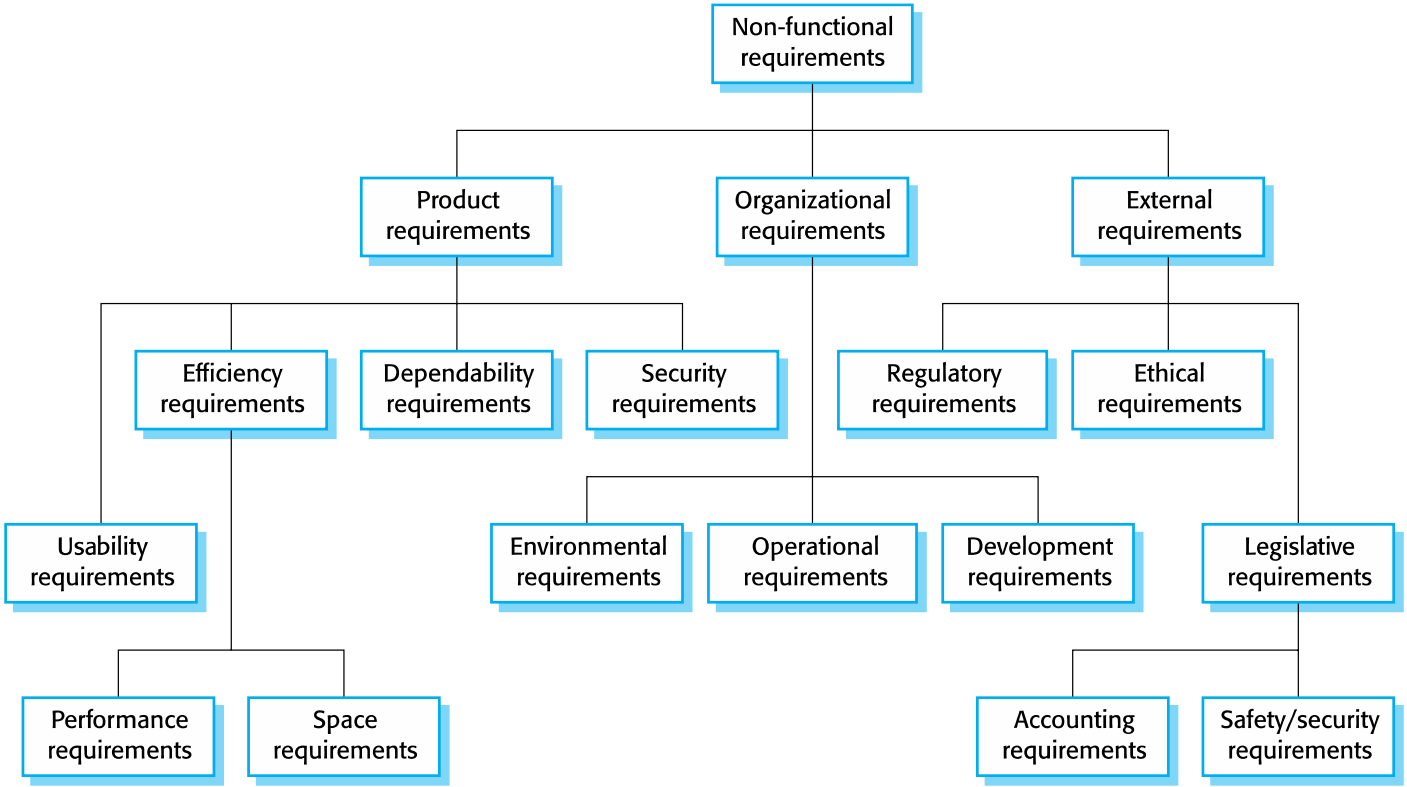
Requirements Completeness and Consistency

- In principle, requirements should be both complete and consistent.
- **Complete**
  - Should include descriptions of all facilities required.
- **Consistent**
  - There should be no conflicts or contradictions in the descriptions of the system facilities.
- In practice, it is impossible to produce a complete and consistent requirements document.

Non-Functional Requirements

- These define system properties (e.g. reliability, response time, storage requirements) and constraints (e.g. I/O device capabilities, system representations, etc.).
- Non-functional requirements may be more critical than functional requirements.
- If these are not met, the system may be useless.
- **Non-Functional Requirements Implementation**
  - Non-functional requirements may affect the overall architecture of a system rather than the individual components.
  - A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.

Types of Non-Functional Requirement



Non-functional requirements branch into product, organisational, and external requirements.

Non-Functional Classifications

- **Product requirements**
  - Requirements which specify that the delivered product must behave in a particular way, e.g. execution speed, reliability, etc.
  - Example: The Mentcare system shall be available to all clinics during normal working hours (Mon–Fri, 08:30–17:30). Downtime shall not exceed five seconds in any one day.
- **Organisational requirements**
  - Requirements which are a consequence of organisational policies and procedures, e.g. process standards, implementation requirements, etc.
  - Example: Users of the Mentcare system shall authenticate themselves using their health authority identity card.
- **External requirements**
  - Requirements which arise from factors external to the system and its development process, e.g. interoperability, legislative requirements, etc.
  - Example: The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Goals and Requirements

- Non-functional requirements may be very difficult to state precisely, and imprecise requirements may be difficult to verify.
- **Goal:** A general intention of the user, such as ease of use.
- **Verifiable non-functional requirement:** A statement using some measure that can be objectively tested.
- Goals are helpful to developers as they convey the intentions of the system users.
- Example: Usability Requirements
  - The system should be easy to use by medical staff and should be organized so that user errors are minimized. (Goal)
  - Medical staff shall be able to use all system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use. (Testable non-functional requirement)

Metrics for Specifying Non-Functional Requirements

Property	Measure
Speed	Processed transactions/second; user/event response time; screen refresh time
Size	Mbytes; number of ROM chips
Ease of use	Training time; number of help frames
Reliability	Mean time to failure; probability of unavailability; rate of failure occurrence; availability
Robustness	Time to restart after failure; % of events causing failure; probability of data corruption on failure
Portability	% of target dependent statements; number of target systems

Topics Covered

1. Functional and non-functional requirements
2. **Requirements engineering processes**
3. Requirements elicitation and analysis
4. Requirements specification
5. Requirements validation
6. Requirements management

Requirements Engineering Processes

- RE processes depend on the application domain, the people involved, and the organisation developing the requirements.
- **Generic activities common to all processes:**
  - Requirements elicitation;
  - Requirements analysis;
  - Requirements specification;
  - Requirements validation;
  - Requirements management.
- **In practice, RE is an iterative activity**
  - These activities are interleaved.

Topics Covered

1. Functional and non-functional requirements
2. Requirements engineering processes
3. **Requirements elicitation and analysis**
4. Requirements specification
5. Requirements validation
6. Requirements management

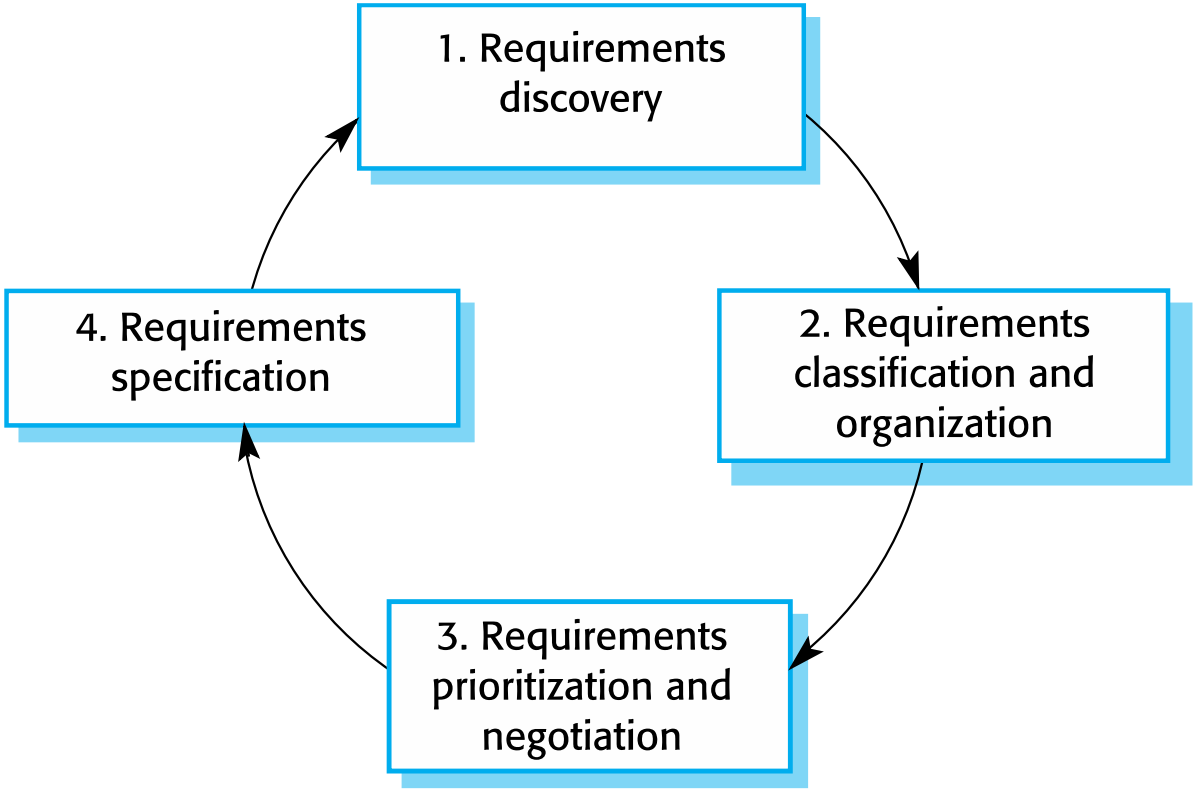
Requirements Elicitation and Analysis

- Sometimes called requirements elicitation or requirements discovery.
- Software engineers work with a range of system stakeholders to find out about:
  - the application domain,
  - the services that the system should provide,
  - the required system performance, hardware constraints, other systems, etc.
- **May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc.** These are called stakeholders.

Problems of Requirements Elicitation and Analysis

- Stakeholders don’t know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.

Requirements Elicitation and Analysis Process



The four activities are interleaved in an iterative cycle: discovery → classification → prioritisation → specification.

Process Activities

- **Requirements discovery**
  - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- **Requirements classification and organisation**
  - Groups related requirements and organises them into coherent clusters.
- **Prioritisation and negotiation**
  - Prioritising requirements and resolving requirements conflicts.
- **Requirements specification**
  - Requirements are documented and input into the next round of the spiral.

Requirements Discovery

- The process of gathering information about the required and existing systems and distilling the user and system requirements from this information.
- Interaction is with system stakeholders, from managers to external regulators.
- Systems normally have a range of stakeholders.

Interviewing

- Formal or informal interviews with stakeholders are part of most RE processes.
- **Types of interview**
  - Closed interviews based on a pre-determined list of questions.
  - Open interviews where various issues are explored with stakeholders.

Interviews in Practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviewers need to be open-minded, without pre-conceived ideas of what the system should do.
- You need to prompt users to talk about the system by suggesting requirements rather than simply asking them what they want.

Problems with Interviews

- Application specialists may use language to describe their work that isn't easy for the requirements engineer to understand.
- **Interviews are not good for understanding domain requirements**
  - Requirements engineers cannot understand specific domain terminology;
  - Some domain knowledge is so familiar that people find it hard to articulate, or think it isn't worth articulating.

Ethnography

- A social scientist spends a considerable time observing and analysing how people actually work.
- People do not have to explain or articulate their work.
- Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

Stories and Scenarios

- Scenarios and user stories are real-life examples of how a system can be used.
- Stories and scenarios are a description of how a system may be used for a particular task.
- Because they are based on a practical situation, stakeholders can relate to them and can comment on their situation with respect to the story.

Photo Sharing in the Classroom (iLearn)

Jack is a primary school teacher in Ullapool (a village in northern Scotland). He has decided that a class project should be focused around the fishing industry in the area, looking at the history, development and economic impact of fishing. As part of this, pupils are asked to gather and share reminiscences from relatives, use newspaper archives and collect old photographs related to fishing and fishing communities in the area. Pupils use an iLearn wiki to gather together fishing stories and SCRAN (a history resources site) to access newspaper archives and photographs. However, Jack also needs a photo sharing site as he wants pupils to take and comment on each others' photos and to upload scans of old photographs that they may have in their families.

Jack sends an email to a primary school teachers group, which he is a member of to see if anyone can recommend an appropriate system. Two teachers reply and both suggest that he uses KidsTakePics, a photo sharing site that allows teachers to check and moderate content. As KidsTakePics is not integrated with the iLearn authentication service, he sets up a teacher and a class account. He uses the iLearn setup service to add KidsTakePics to the services seen by the pupils in his class so that when they log in, they can immediately use the system to upload photos from their mobile devices and class computers.

Scenarios

- A structured form of user story.
- **Scenarios should include:**
  - A description of the starting situation;
  - A description of the normal flow of events;
  - A description of what can go wrong;
  - Information about other concurrent activities;
  - A description of the state when the scenario finishes.

Uploading Photos [1]

- **Initial assumption:** A user or group of users have one or more digital photographs to be uploaded to the picture sharing site, saved on a tablet or laptop, and have successfully logged on to KidsTakePics.
- **Normal:** The user chooses to upload photos, selects them on their computer, and selects the project name under which they will be stored. They may also input keywords for each photo. Uploaded photos are named by combining the user name with the local filename.
- On completion, the system automatically emails the project moderator to check new content, and shows an on-screen message to the user confirming this has been done.

Uploading Photos [2]

- **What can go wrong:**
  - No moderator is associated with the selected project — the school administrator is emailed to nominate one; users are informed of a possible delay.
  - Photos with the same name have already been uploaded by the same user — the user is asked to re-upload (overwriting), rename (auto-numbered), or cancel.
- **Other activities:** The moderator may be logged on and may approve photos as they are uploaded.
- **System state on completion:** Selected photos are uploaded with status ‘awaiting moderation’, visible to the moderator and the uploader.

Topics Covered

1. Functional and non-functional requirements
2. Requirements engineering processes
3. Requirements elicitation and analysis
4. **Requirements specification**
5. Requirements validation
6. Requirements management

“I know you think you understand what I said, but what you don’t understand is what I said is not what I meant.”



Requirements Specification

- The process of writing down the user and system requirements in a requirements document.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed and may include more technical information.
- May be part of a contract for the system development.
- Should therefore be a complete and detailed specification of the whole system.

Requirements and Design

- **In principle:**
  - requirements should state what the system should do, and
  - the design should describe how it does this.
- **In practice: requirements and design are inseparable**
  - A system architecture may be designed to structure the requirements;
  - The system may inter-operate with other systems that generate design requirements;
  - Using a specific architecture to satisfy non-functional requirements may be a domain requirement;
  - This may be the consequence of a regulatory requirement.

Ways of Writing a System Requirements Specification

1. Natural language
2. Structured natural language
3. Design description languages
4. Graphical notations
5. Mathematical specifications

Natural Language Specification

- Requirements are written as natural language sentences, supplemented by diagrams and tables.
- Used for writing requirements because it is expressive, intuitive, and universal.
- This means the requirements can be understood by users and customers.

Guidelines for Writing Requirements

- Invent a standard format and use it for all requirements.
- Use language in a consistent way.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.

Problems with Natural Language

- Lack of clarity
- Requirements confusion
- Requirements amalgamation

Example: Requirements for the Insulin Pump

- **3.2** The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (Changes in blood sugar are relatively slow, so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)
- **3.6** The system shall run a self-test routine every minute, with the conditions to be tested and the associated actions defined. (A self-test routine can discover hardware and software problems and alert the user that normal operation may be impossible.)

Structured Specifications

- The freedom of the requirements writer is limited, and requirements are written in a standard way.
- This approach maintains most of the expressiveness and understandability of natural language but ensures some uniformity is imposed on the specification.
- This works well for some types of requirements, e.g. embedded control systems, but is sometimes too rigid for writing business system requirements.

Structured Specification — Insulin Pump [1]

Insulin Pump / Control Software / SRS / 3.3.2

Field	Detail
Function	Compute insulin dose: safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2); the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose — the dose of insulin to be delivered.
Destination	Main control loop.

Structured Specification — Insulin Pump [2]

Field	Detail
Action	CompDose is zero if the sugar level is stable or falling, or increasing with decreasing rate. If increasing with increasing rate, CompDose is computed by dividing the difference between the current and previous level by 4 and rounding. If rounded to zero, CompDose is set to the minimum dose.
Requirements	Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Post-condition	r0 is replaced by r1, then r1 is replaced by r2.
Side effects	None.

Tabular Specification

- Used to supplement natural language.
- Particularly useful when you have to define a number of possible alternative courses of action.
- **For example:**
  - the insulin pump system bases its computations on the rate of change of blood sugar level, and the tabular specification explains how to calculate the insulin requirement for different scenarios.

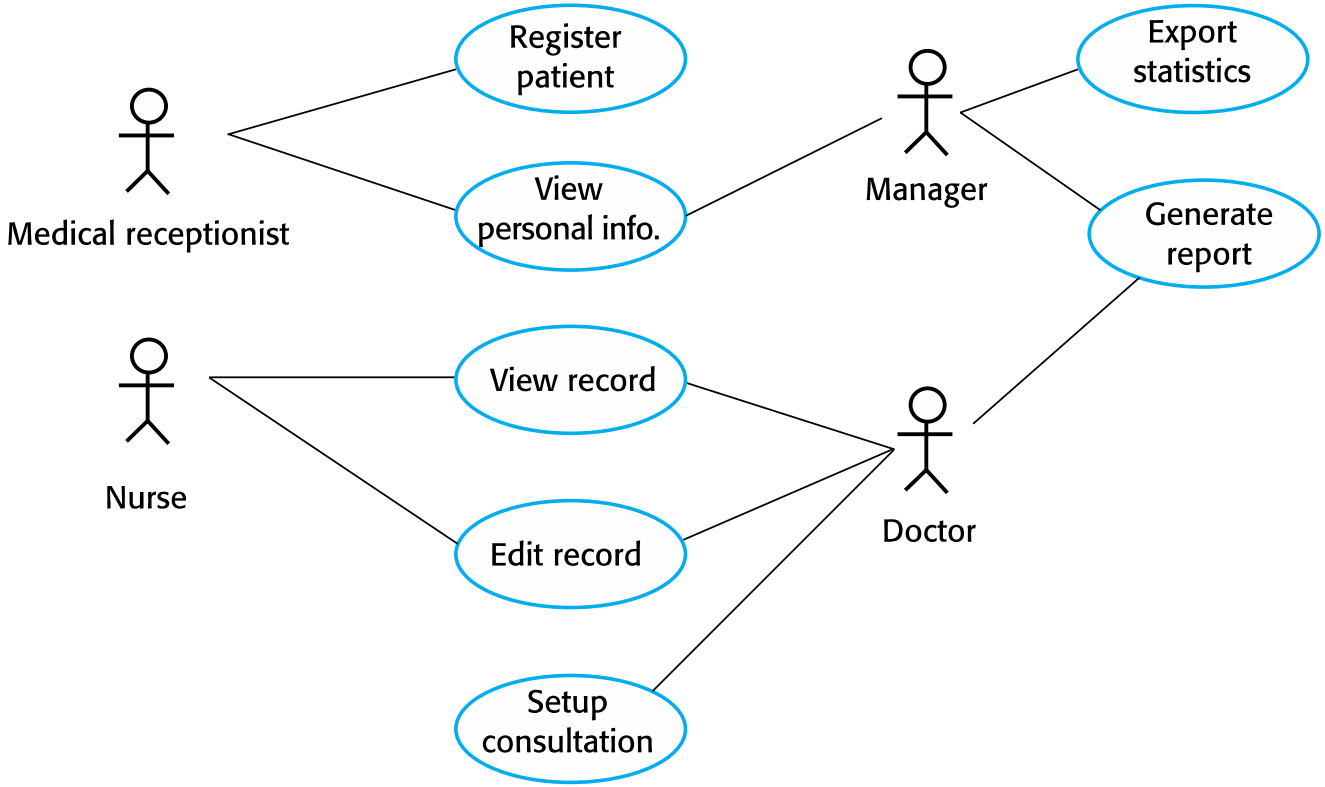
Tabular Specification of Computation for an Insulin Pump

Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Increasing, rate decreasing ((r2−r1) < (r1−r0))	CompDose = 0
Increasing, rate stable/increasing ((r2−r1) ≥ (r1−r0))	CompDose = round((r2−r1)/4); if 0, CompDose = MinimumDose

Use Cases

- Use cases are a kind of scenario that are included in the UML.
- Use cases identify the actors in an interaction and describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- High-level graphical model, supplemented by a more detailed tabular description.
- UML sequence diagrams may be used to add detail to use cases by showing the sequence of event processing in the system.

Use Cases for the Mentcare System



Medical receptionist, nurse, manager, and doctor each interact with a distinct set of use cases.

The Software Requirements Document

- The software requirements document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- **It is NOT a design document.** As far as possible, it should set out WHAT the system should do rather than HOW it should do it.

Requirements Document Variability

- Information in the requirements document depends on the type of system and the approach to development used.
- Systems developed incrementally will typically have less detail in the requirements document.
- Requirements document standards have been designed, e.g. the IEEE standard. These are mostly applicable to large systems engineering projects.

Structure of a Requirements Document [1]

Chapter	Description
Preface	Defines the expected readership, version history, rationale for the new version, and a summary of changes made in each version.
Introduction	Describes the need for the system, its functions, how it works with other systems, and how it fits into the organization's strategic objectives.
Glossary	Defines the technical terms used in the document, without assuming reader expertise.
User requirements definition	Describes services provided for the user, using natural language, diagrams, or other accessible notations. Product and process standards must be specified.
System architecture	Presents a high-level overview of the anticipated system architecture and the distribution of functions across modules.

Structure of a Requirements Document [2]

Chapter	Description
System requirements specification	Describes functional and non-functional requirements in more detail, including interfaces to other systems.
System models	Graphical models showing relationships between system components and their environment, e.g. object, data-flow, or semantic data models.
System evolution	Describes fundamental assumptions and anticipated changes due to hardware evolution or changing user needs.
Appendices	Detailed application-specific information, e.g. hardware and database descriptions.
Index	Alphabetic index, plus indexes of diagrams, functions, and so on.

Topics Covered

1. Functional and non-functional requirements
2. Requirements engineering processes
3. Requirements elicitation and analysis
4. Requirements specification
5. **Requirements validation**
6. Requirements management

Requirements Validation

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high, so validation is very important.
- Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements Checking

- **Validity** — Does the system provide the functions which best support the customer's needs?
- **Consistency** — Are there any requirements conflicts?
- **Completeness** — Are all functions required by the customer included?
- **Realism** — Can the requirements be implemented given available budget and technology?
- **Verifiability** — Can the requirements be checked?

Requirements Validation Techniques

- **Requirements reviews**
  - Systematic manual analysis of the requirements.
- **Prototyping**
  - Using an executable model of the system to check requirements.
- **Test-case generation**
  - Developing tests for requirements to check testability.

Requirements Reviews

- Regular reviews should be held while the requirements definition is being formulated.
- Both client and contractor staff should be involved in reviews.
- Reviews may be formal (with completed documents) or informal. Good communication between developers, customers, and users can resolve problems at an early stage.

Review Checks

- **Verifiability** — Is the requirement realistically testable?
- **Comprehensibility** — Is the requirement properly understood?
- **Traceability** — Is the origin of the requirement clearly stated?
- **Adaptability** — Can the requirement be changed without a large impact on other requirements?

Topics Covered

1. Functional and non-functional requirements
2. Requirements engineering processes
3. Requirements elicitation and analysis
4. Requirements specification
5. Requirements validation
6. **Requirements management**

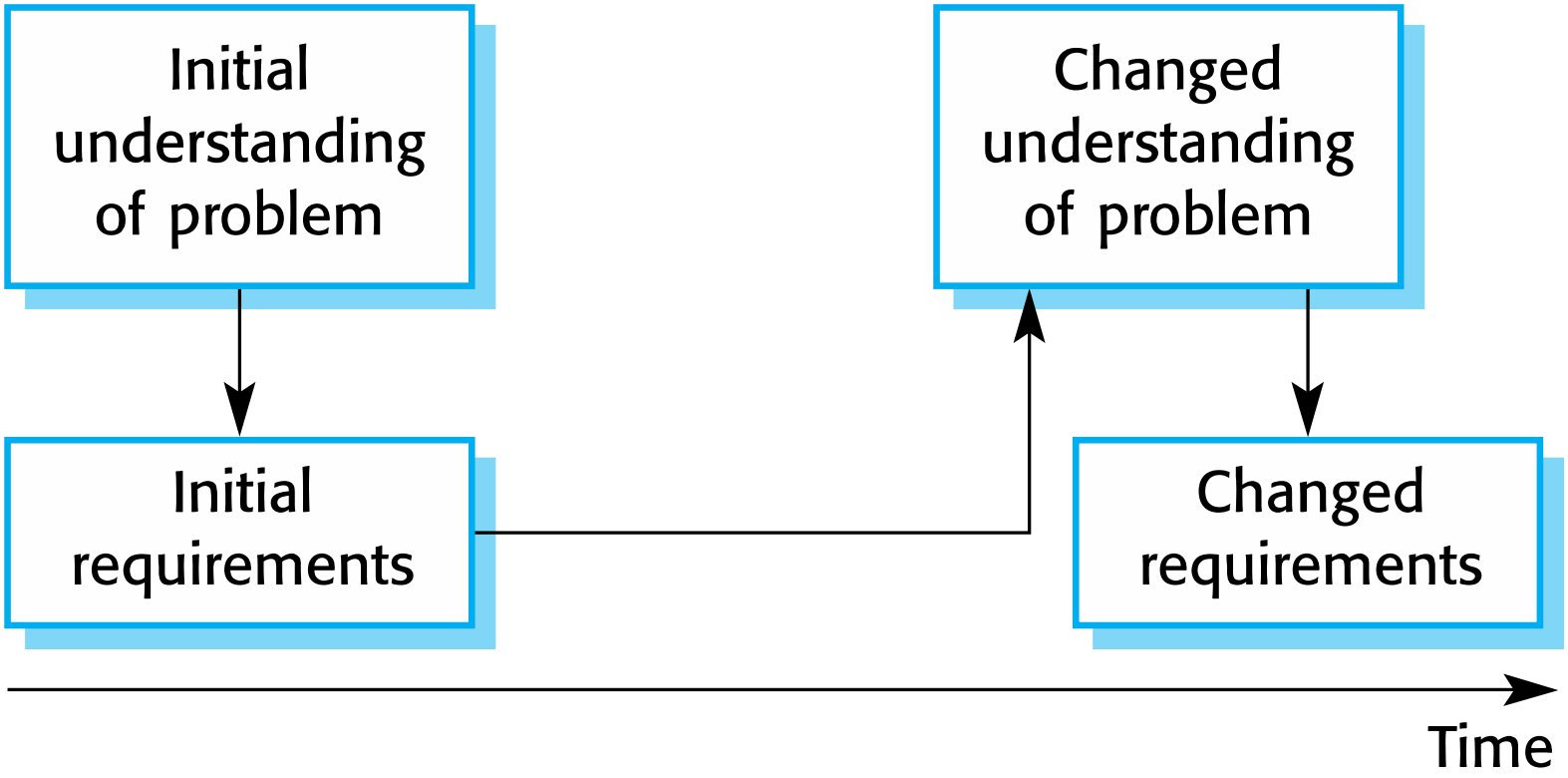
Changing Requirements

- The business and technical environment of the system always changes after installation.
- The people who pay for a system and the users of that system are rarely the same people.
- Large systems usually have a diverse user community, with many users having different requirements and priorities that may conflict or contradict.

Requirements Management

- Is the process of managing changing requirements during the requirements engineering process and system development.
- **You need to:**
  - keep track of individual requirements and maintain links between dependent requirements, so you can assess the impact of changes;
  - establish a formal process for making change proposals and linking these to system requirements.

Requirements Evolution

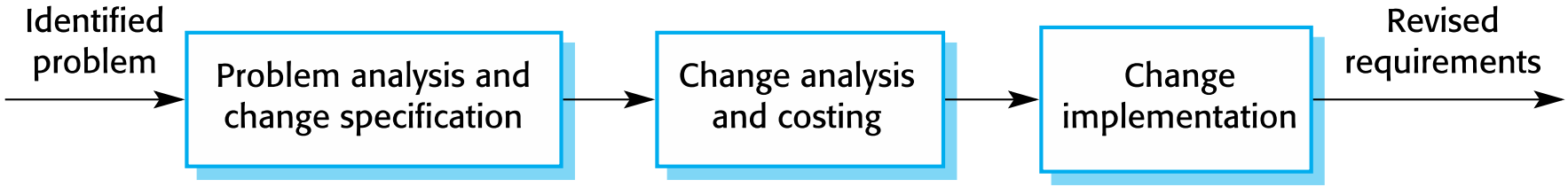


As understanding of the problem changes over time, requirements must change to match.

Requirements Management Planning

- Planning is an essential first stage; it establishes the level of requirements management detail that is required.
- **During requirements management planning, decide on:**
  - **Requirements identification** — each requirement must be uniquely identified so it can be cross-referenced with other requirements.
  - **A change management process** — the set of activities that assess the impact and cost of changes.
  - **Traceability policies** — define the relationships between requirements, and between requirements and the system design, that should be recorded.
  - **Tool support** — ranges from specialist requirements management systems to spreadsheets and simple databases.

Requirements Change Management



An identified problem flows through change specification, analysis and costing, and implementation to produce revised requirements.

Questions?

Reading: Sommerville, Software Engineering, 10th Ed — Chapter 4

Nguyễn Thị Minh Tuyền · Faculty of IT, HCMUS

CS300· Elements of Software Engineering · Week 4